

부산 수학문화관 인터랙티브 교육 키오스크

Unity 6 · C# · FSM 아키텍처 · 터치 키오스크 고대 문명의 숫자 체계를 게임·영상·드로잉으로 체험하는 박물관 전시 콘텐츠

프로젝트 요약

항목	내용
프로젝트명	부산 수학문화관 인터랙티브 교육 애플리케이션
개발 기간	2026.03
플랫폼	Windows (터치 디스플레이 키오스크)
엔진	Unity 6 (URP)
언어	C#
주요 역할	클라이언트 전체 설계 및 개발
배포 환경	부산 수학문화관 현장 키오스크

프로젝트 소개

부산 수학문화관에 설치되는 터치 키오스크 전시물로, 이집트·중국·로마 3개 문명의 역사적 숫자 체계를 체험 학습할 수 있는 인터랙티브 교육 애플리케이션입니다. 관람객이 직접 터치하며 고대 숫자를 배우고, 게임으로 학습하고, 직접 그려보는 경험을 제공합니다.

-  문명별 교육 영상으로 역사적 숫자 체계 소개
-  숫자 변환 퀴즈로 학습 내용 확인
-  카드 매칭 게임으로 숫자 기억력 테스트
-  자신의 생일을 고대 숫자로 변환하여 직접 드로잉
-  가장 마음에 드는 숫자 체계에 투표

박물관 전시 환경에 맞춘 무인 키오스크 설계로, 60초 무입력 시 자동 홈 복귀 등 무인 운영이 가능합니다.

해결하려는 문제

문제	설명
수동적 관람 경험	기존 전시물은 패널/영상 시청에 그쳐 관람객 참여도가 낮음
11개 화면의 복잡한 전환	영상 → 게임 → 투표 → 드로잉 등 비선형 흐름에서 상태 관리가 복잡
무인 키오스크 안정성	관리자 없이 장시간 운영 시 리소스 누수, 이전 사용자 상태 잔존 위험
비활성 UI 초기화 누락	Unity의 비활성 GameObject는 Awake/Start가 호출되지 않아 초기화 시점 문제 발생

해결 전략:

- FSM(유한 상태 머신)으로 11개 화면의 생명주기를 체계적으로 관리
- 5단계 생명주기(Init/Enter/Update/Exit/Dispose)로 리소스 누수 방지
- IdleManager로 60초 무입력 감지 시 자동 홈 복귀
- EnsureInitialized 패턴으로 비활성 View도 안전하게 초기화

핵심 기능

인트로 · 교육 영상

문명별 소개 영상 + 진행바 드래그/스킵 지원, 90% 이상 시청 시 국가 선택 버튼 자동 등장

숫자 변환 퀴즈

랜덤 출제된 아라비아 숫자를 이집트 상형문자/중국 한자/로마 숫자로 변환하는 퀴즈

60초 카드 매칭 게임

6쌍(12장) 카드를 셔플하여 제한 시간 내 매칭 — DOTween Y축 회전 플립 애니메이션

투표 시스템

가장 마음에 드는 숫자 체계에 투표, JSON 파일로 영속화하여 누적 결과를 순위/바 차트로 표시

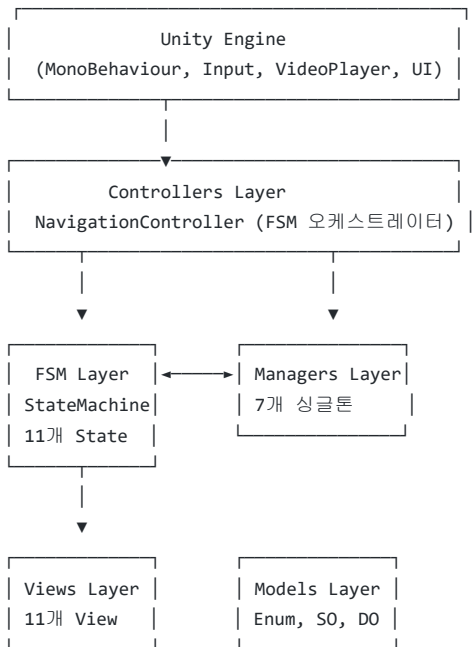
생일 드로잉

년/월/일을 선택하면 해당 문명의 숫자 이미지로 변환된 미리보기를 반투명 오버레이로 표시, 터치로 직접 따라 그리기

자동 홈 복귀

60초 무입력 시 자동으로 홈 화면 복귀 — 무인 키오스크 연속 운영 보장

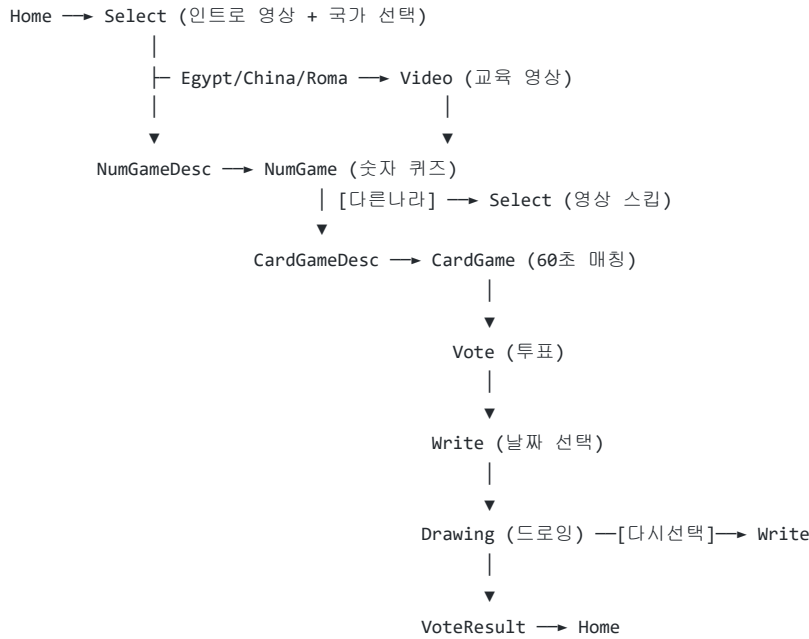
시스템 아키텍처



설계 목표:

- State-View 1:1 매핑으로 화면별 책임 분리
- Manager 싱글톤으로 게임/영상/투표 등 도메인 로직 캡슐화
- 이벤트 기반(Action 델리게이트) 느슨한 결합으로 모듈 간 의존성 최소화
- NavigationController가 유일한 화면 전환 진입점 → 흐름 추적 용이

화면 흐름



UX 특징:

- 어떤 화면에서든 홈 버튼으로 즉시 복귀 가능
- 60초 무입력 시 자동 홈 복귀 (무인 운영)
- 다른 나라 선택 시 인트로 영상 95% 스킵 → 바로 국가 버튼 표시
- 영상 진행바 되감기 시 버튼 페이드아웃, 재도달 시 페이드인

핵심 설계

FSM (유한 상태 머신)

11개 화면이 순차/분기적으로 전환되며, 각 화면마다 독립적인 진입/이탈 로직이 필요하여 FSM을 적용했습니다.

IState: Init → Enter → Update(매 프레임) → Exit → Dispose
↑ 1회만 ↑ 매 전환마다 ↑ 앱 종료

StateMachine: HashSet으로 Init 1회 호출 보장
Dictionary<Type, IState>로 상태 관리

장점:

- 5단계 생명주기로 리소스 할당/해제 시점이 명확
- HashSet 기반 Init 추적으로 이벤트 중복 구독 방지
- State가 순수 C# 클래스 → Unity 생명주기에 비의존, 초기화 순서 완전 제어

MVC 변형 (State-View 패턴)

View는 UI 요소 참조와 이벤트 발생만 담당하고, 비즈니스 로직은 State에서 처리합니다.

View: Button.onClick → Action 이벤트 발생
State: Action 구독 → Manager 호출 → View UI 업데이트

장점:

- UI 변경이 로직에 영향 없음

- State 단위 테스트 용이
- 11개 화면을 일관된 패턴으로 구현

EnsureInitialized 패턴

Unity에서 비활성 GameObject는 Awake/Start가 호출되지 않는 문제를 해결합니다.

```
앱 시작 → NavigationController.EnsureAllViewsInitialized()
        ↳ 11개 View.EnsureInitialized() (활성/비활성 무관)
            ↳ Initialize() + BindUIEvent()
```

장점:

- 비활성 상태로 시작하는 View도 확실하게 초기화
- 초기화 순서를 명시적으로 제어

기술 스택

분류	기술
게임 엔진	Unity 6 (URP)
언어	C#
UI	Unity UI (uGUI), TextMeshPro
영상	VideoPlayer + RenderTexture
애니메이션	DOTween
드로잉	DrawTextureUI (외부 에셋)
데이터	JSON (JsonUtility), ScriptableObject
이벤트	System.Action 델리게이트

⚙️ 주요 시스템

NavigationController

FSM 오케스트레이터 — 11개 GoToXxx() 메서드로 모든 화면 전환을 중앙 관리

StateMachine

Dictionary + HashSet 기반 상태 관리 — ChangeState()로 상태 전환, Init 1회 보장

IdleManager

마우스/터치/키보드 입력을 매 프레임 감지하여 60초 무입력 시 자동 홈 복귀

VideoManager

VideoPlayer 래핑 — 재생/정지/스킵/진행률 조회, RenderTexture 기반 영상 출력

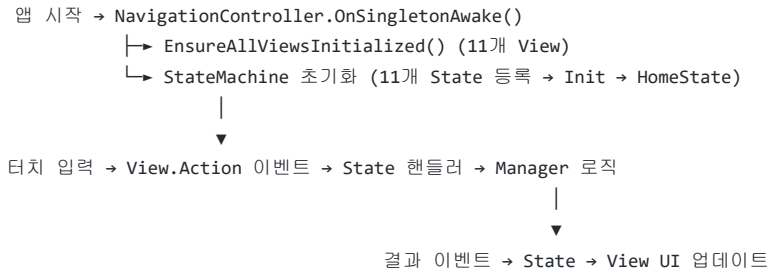
VoteManager

투표 데이터 JSON 영속화 — 앱 재시작 후에도 누적 투표 결과 유지

CardGameManager

6쌍 카드 셔플/매칭 판정 — Action 이벤트로 성공/실패/클리어 상태 전파

데이터 흐름



기술적 도전

1 비활성 GameObject 초기화 문제

문제점

Unity에서 처음부터 비활성화된 11개 화면의 View는 Awake/Start가 호출되지 않아, 이벤트 바인딩이 누락되는 문제가 발생했습니다.

해결

EnsureInitialized 패턴을 설계하여, 앱 시작 시 UINavigationController가 모든 View를 명시적으로 초기화합니다. bool 플래그로 중복 초기화를 방지하고, BaseView 추상 클래스에 패턴을 내장하여 모든 View가 일관된 초기화 흐름을 따르도록 했습니다.

2 11개 화면의 복잡한 상태 전환 관리

문제점

순차 흐름 + 분기(다른 나라 선택, 다시 선택) + 예외(60초 무입력 복귀)가 혼합된 복잡한 전환 흐름을 안정적으로 관리해야 했습니다.

해결

FSM에 5단계 생명주기(Init/Enter/Update/Exit/Dispose)를 설계하여 각 상태의 진입·이탈 시점에 리소스를 확실히 관리합니다. UINavigationController를 유일한 전환 진입점으로 설계하여 전환 흐름을 한 곳에서 파악할 수 있게 했습니다. HashSet으로 Init 1회 호출을 보장하여 이벤트 중복 구독 문제를 원천 차단했습니다.

3 무인 키오스크 안정성

문제점

이전 사용자의 게임/영상 상태가 다음 사용자에게 잔존하거나, 장시간 운영 시 리소스 누수가 발생할 위험이 있었습니다.

해결

IdleManager가 매 프레임 마우스/터치/키보드 입력을 감지하여 60초 무입력 시 홈으로 강제 복귀합니다. 각 State의 Exit()에서 Manager 리셋, 코루틴 정지, 애니메이션 킬 등 리소스를 확실히 정리하도록 설계하여 상태 잔존을 방지했습니다.

설계 트레이드오프

선택	이유
State를 순수 C# 클래스로 구현 (MonoBehaviour X)	Unity 생명주기에 비의존 → 초기화 순서 완전 제어, 코루틴은 Manager에 위임
View의 UI 요소를 public 필드로 노출	11화면 규모에서 과도한 캡슐화보다 개발 속도와 Inspector 편의 우선
NavigationController만 ChangeState 호출	전환 로직 중앙 집중으로 흐름 추적 용이, 분산 호출 시 디버깅 난이도 증가 방지
국가별 스프라이트를 List로 관리	년(1980~2030)/월(1~12)/일(1~31) 모두 연속 범위이므로 Dictionary보다 List가 최적
영상 스킵 시 95% 지점으로 이동	완전 스킵 대신 자연스러운 영상 종료 콜백 활용, 기존 종료 로직 재사용

프로젝트 결과

- 이집트·중국·로마 3개 문명의 숫자 체계를 영상/게임/드로잉으로 체험하는 11개 화면 키오스크 콘텐츠 완성
- FSM 기반 5단계 생명주기로 화면 전환 시 리소스 누수 없는 안정적 운영
- 60초 무입력 자동 복귀로 무인 키오스크 연속 운영 가능
- EnsureInitialized 패턴으로 Unity 비활성 GameObject 초기화 문제 해결
- 투표 데이터 JSON 영속화로 누적 관람객 통계 수집

배운 점

- FSM의 생명주기를 세분화(Init/Dispose 분리)하면 이벤트 구독/해제를 체계적으로 관리할 수 있다
- Unity의 비활성 GameObject 초기화 문제는 프레임워크 레벨에서 패턴으로 해결해야 누락이 없다
- 키오스크처럼 무인 운영되는 환경에서는 "정리"가 "기능"만큼 중요하다 — Exit()의 확실한 리소스 해제가 안정성의 핵심
- NavigationController에 전환 로직을 집중시키면 복잡한 화면 흐름도 한눈에 파악 가능

향후 개선

- 드로잉 결과물 인쇄 기능 연동 (키오스크 프린터)
- 다국어(영어/일본어) 지원으로 외국인 관람객 대응